WEEK 14

Testing & Debugging

Daily study material with detailed examples

Android Development with Kotlin | Prepared for **Anjana**

Professional developers don't just write code: they **prove it works** with automated tests, and they know how to find bugs fast. In a team, pull requests usually need tests, and CI runs them on every change. This week you'll write **unit tests** for logic, repositories and ViewModels, get to know **Compose UI tests**, and practise **debugging**. You'll finish by writing a test suite for a Notes app.

Day	Topic	You will be able to...
Day 1	Why test? Your first unit test	Write and run a JUnit test
Day 2	Writing good tests	Name tests well, cover edge cases, use Arrange-Act-Assert
Day 3	Fakes and repository tests	Test code that depends on databases and APIs
Day 4	Testing ViewModels	Test state changes with coroutines and a MainDispatcherRule
Day 5	UI tests and debugging	Write Compose UI tests and use the debugger
Day 6	Weekly project	Write a test suite for the Notes app

Tip: The unit tests in this book were **executed**: the logic and every assertion really ran, and the "Test results" shown are the real results. JUnit and kotlinx-coroutines-test couldn't be downloaded here, so a small runner with the same API (@Test, assertEquals, runTest, setMain) ran them. In Android Studio, the real libraries give the same results. Compose UI tests are shown for reference.

WEEK 14 • DAY 1

Why Test? Your First Unit Test

Today's goal: Understand the kinds of tests, where they live in a project, and write and run your first JUnit test.

1. Why write tests?

- **Catch bugs early**, before users (or your team lead) find them.
- **Change code without fear**: if you break something, a test fails immediately.
- **Document behaviour**: a test shows exactly what a function is supposed to do.
- **Teams require it**: CI runs all tests on every pull request, and a red test blocks the merge.

2. Kinds of tests

Sketch: The testing pyramid

```
              /\
             /UI\          Few: slow, run on a device/emulator (androidTest/)
            /----\
           /Integr.\       Some: several real parts together
          /--------\
         /   Unit   \      Many: fast, run on your computer in milliseconds (test/)
        /------------\
```

Folder	Type	Runs on	Tests
app/src/**test**/java/...	Local unit tests	Your computer (JVM): fast	Logic, ViewModels, repositories with fakes
app/src/**androidTest**/java/...	Instrumented tests	A phone or emulator: slower	UI tests, Room database tests

Test dependencies (Android only; usually already in the template)

```
dependencies {
    testImplementation("junit:junit:4.13.2")
    testImplementation("org.jetbrains.kotlinx:kotlinx-coroutines-test:1.9.0")
    androidTestImplementation("androidx.test.ext:junit:1.2.1")
    androidTestImplementation("androidx.compose.ui:ui-test-junit4")      // Compose UI tests
    debugImplementation("androidx.compose.ui:ui-test-manifest")
}
```

3. The code we'll test

Testing is easiest for **pure functions**: the same input always gives the same output, with no Android, network or database.

4. Your first test

PriceCalculatorTest.kt (in app/src/test/...)

```
import org.junit.Assert.assertEquals
import org.junit.Test

// --- the code under test (normally in app/src/main/...) ---
fun finalPrice(price: Int, discountPercent: Int): Int {
    require(discountPercent in 0..100) { "Invalid discount" }
    return price - price * discountPercent / 100
}

// --- the test class ---
class PriceCalculatorTest {

    @Test
    fun noDiscount_returnsSamePrice() {
        assertEquals(500, finalPrice(500, 0))
    }

    @Test
    fun tenPercentDiscount_reducesPrice() {
        assertEquals(450, finalPrice(500, 10))
    }

    @Test
    fun fullDiscount_returnsZero() {
        assertEquals(0, finalPrice(500, 100))
    }
}
```

Output

```
PriceCalculatorTest
  PASSED  fullDiscount_returnsZero
  PASSED  noDiscount_returnsSamePrice
  PASSED  tenPercentDiscount_reducesPrice
3 tests, 0 failed
```

Code	Meaning
class PriceCalculatorTest	A test class. Name it after the class or file under test + Test
@Test	Marks a function as a test. JUnit runs every @Test function
assertEquals(expected, actual)	Fails the test if the two values are different. **Expected comes first**

5. Running tests in Android Studio

- Click the green ■ next to a test function (one test) or the class name (all tests in the class).
- Or right-click the **test** folder → **Run 'Tests in ...'** to run everything.
- Results appear in the **Run** window: green tick = passed, red cross = failed, with the expected and actual values.
- From the terminal: `./gradlew test`. This is what CI runs.

6. What a failing test looks like

A bug caught by a test

```
import org.junit.Assert.assertEquals
import org.junit.Test

fun buggyAverage(marks: List<Int>): Int = marks.sum() / (marks.size + 1)     // bug: + 1

class AverageTest {
    @Test
    fun averageOfThreeMarks() {
        assertEquals(80, buggyAverage(listOf(70, 80, 90)))
    }
}
```

Output

```
AverageTest
  FAILED  averageOfThreeMarks: expected:<80> but was:<60>
1 tests, 1 failed
```

The message **expected:<80> but was:<60>** tells you exactly what went wrong. Fix the code (remove the + 1), run again, and the test turns green. That's the daily rhythm of testing.

Practice Exercises

1. Write tests for **fun isEven(n: Int): Boolean**: one for an even number, one for an odd number, and one for 0.

2. Write tests for **fun celsiusToFahrenheit(c: Double): Double** using assertEquals(expected, actual, 0.01) for 0, 100 and 37.

3. Introduce a bug on purpose in one function, run its tests, read the failure message, then fix it.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Where do local unit tests live?

Q2. What is the difference between test/ and androidTest/?

Q3. What does @Test do?

Q4. In assertEquals, which value comes first?

Q5. Which command runs all unit tests from the terminal?

WEEK 14 • DAY 2

Writing Good Tests

Today's goal: Write clear, focused tests with good names, the Arrange-Act-Assert pattern and edge cases.

1. Arrange, Act, Assert

Every good test has three parts:

Part	What	Example
Arrange	Prepare the inputs and objects	val cart = Cart(); cart.add(item)
Act	Do the one thing you're testing	val total = cart.total()
Assert	Check the result	assertEquals(150, total)

2. Names that explain

When a test fails in CI, its **name** is the first thing you see. Kotlin lets you use readable names in backticks for unit tests:

Readable test names and AAA

```
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

data class Item(val name: String, val price: Int, val quantity: Int)

class Cart {
    private val items = mutableListOf<Item>()
    fun add(item: Item) { items.add(item) }
    fun total(): Int = items.sumOf { it.price * it.quantity }
    fun isEmpty(): Boolean = items.isEmpty()
}

class CartTest {
    @Test
    fun `new cart is empty`() {
        val cart = Cart()                       // Arrange + Act
        assertTrue(cart.isEmpty())              // Assert
    }

    @Test
    fun `total multiplies price by quantity`() {
        val cart = Cart()                                       // Arrange
        cart.add(Item("Pen", 10, 3))
        cart.add(Item("Notebook", 45, 2))

        val total = cart.total()                                // Act

        assertEquals(120, total)                                // Assert
    }
}
```

Output

```
CartTest
  PASSED  new cart is empty
  PASSED  total multiplies price by quantity
2 tests, 0 failed
```

Tip: Backtick names work in **local unit tests** (test/). In instrumented tests (androidTest/) some Android versions don't allow spaces, so teams use names like `totalMultipliesPriceByQuantity` there. Follow your team's style.

3. Edge cases: where bugs hide

Don't only test the "happy path". Test the **boundaries** and unusual inputs:

Testing a validator thoroughly

```
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
import org.junit.Test

fun isValidPassword(password: String): Boolean =
    password.length >= 8 && password.any { it.isDigit() } && password.any { it.isUpperCase() }

class PasswordValidatorTest {
    @Test fun `valid password passes`() = assertTrue(isValidPassword("Kotlin2026"))
    @Test fun `exactly 8 characters is allowed`() = assertTrue(isValidPassword("Abcdefg1"))
    @Test fun `7 characters is too short`() = assertFalse(isValidPassword("Abcdef1"))
    @Test fun `missing a digit fails`() = assertFalse(isValidPassword("KotlinRocks"))
    @Test fun `missing an uppercase letter fails`() = assertFalse(isValidPassword("kotlin2026"))
    @Test fun `empty password fails`() = assertFalse(isValidPassword(""))
}
```

Output

```
PasswordValidatorTest
  PASSED  7 characters is too short
  PASSED  empty password fails
  PASSED  exactly 8 characters is allowed
  PASSED  missing a digit fails
  PASSED  missing an uppercase letter fails
  PASSED  valid password passes
6 tests, 0 failed
```

Edge case	Example
Empty input	"", emptyList()
Boundaries	exactly 8 characters, 7 characters, 0, 100
Negative or very large numbers	-1, Int.MAX_VALUE
null (for nullable parameters)	getGrade(null)
Duplicates and special characters	"Anjana Anjana", "@#$"

4. Testing exceptions

Expecting a function to throw

```
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

fun finalPrice(price: Int, discountPercent: Int): Int {
    require(discountPercent in 0..100) { "Invalid discount" }
    return price - price * discountPercent / 100
}

class DiscountRulesTest {
    @Test
    fun `discount above 100 throws`() {
        val result = runCatching { finalPrice(500, 150) }
        assertTrue(result.isFailure)
        assertEquals("Invalid discount", result.exceptionOrNull()?.message)
    }
}
```

Output

```
DiscountRulesTest
  PASSED  discount above 100 throws
1 tests, 0 failed
```

JUnit 4 also has `@Test(expected = IllegalArgumentException::class)`, and kotlin.test has `assertFailsWith<IllegalArgumentException> { ... }`. Use whichever your project uses.

5. Rules for good tests

Rule	Why
Test one behaviour per test	When it fails, you know exactly what broke
Tests must not depend on each other	JUnit can run them in any order
No real network, database or clock in unit tests	Unit tests must be fast and give the same result every time
Test behaviour, not private details	Refactoring shouldn't break tests if the behaviour is the same

Practice Exercises

1. Write tests for an email validator: valid, missing @, missing dot after @, contains a space, and empty.

2. Write tests for the Week 6 **calculateTip(amount, tipPercent, roundUp)** including a 0 amount and rounding up.

3. Write a test that checks that **getGrade(150)** (marks above 100) throws, after adding a require to getGrade.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What are the three parts of a well-structured test?

Q2. Why give tests descriptive names?

Q3. Give two examples of edge cases.

Q4. Why shouldn't unit tests use the real network?

Q5. How can you check that a function throws?

WEEK 14 • DAY 3

Fakes and Repository Tests

Today's goal: Test code that depends on databases and APIs by replacing them with fakes, and test suspend functions with runTest.

1. The problem: real dependencies

A repository talks to Room and Retrofit. In a unit test you don't want a real database or a real server. You replace them with **fakes**: simple implementations of the same **interface** that keep data in memory. That's why Weeks 11 and 12 used interfaces!

Test double	What it is	Example
Fake	A working, simplified implementation	FakeNoteDao storing notes in a list
Stub	Returns fixed answers	An API that always returns the same 2 users
Mock	Records calls so you can verify them (usually with a library like MockK)	"was delete(5) called once?"

Tip: Google recommends **fakes** over mocks for most Android tests: they're simpler and behave like the real thing.

2. A repository and its fake data sources

UserRepositoryTest with a fake API and a fake DAO

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

// --- production code ---
data class User(val id: Int, val name: String)
interface UserApi { suspend fun fetchUsers(): List<User> }
interface UserDao {
    fun observeAll(): Flow<List<User>>
    suspend fun replaceAll(users: List<User>)
}
class UserRepository(private val api: UserApi, private val dao: UserDao) {
    val users: Flow<List<User>> = dao.observeAll()
    suspend fun refresh(): Result<Unit> = runCatching { dao.replaceAll(api.fetchUsers()) }
}

// --- test doubles ---
class FakeUserApi(var users: List<User> = emptyList(), var shouldFail: Boolean = false) : UserApi {
    override suspend fun fetchUsers(): List<User> {
        if (shouldFail) throw java.io.IOException("No internet")
        return users
    }
}
class FakeUserDao : UserDao {
    private val table = MutableStateFlow<List<User>>(emptyList())
    override fun observeAll(): Flow<List<User>> = table
    override suspend fun replaceAll(users: List<User>) { table.value = users }
}

// --- tests ---
```

```
class UserRepositoryTest {
    @Test
    fun `refresh saves users from the api`() = runTest {
        val api = FakeUserApi(users = listOf(User(1, "Anjana"), User(2, "Rahul")))
        val repository = UserRepository(api, FakeUserDao())

        val result = repository.refresh()

        assertTrue(result.isSuccess)
        assertEquals(listOf("Anjana", "Rahul"), repository.users.first().map { it.name })
    }

    @Test
    fun `refresh failure keeps old data`() = runTest {
        val api = FakeUserApi(users = listOf(User(1, "Anjana")))
        val repository = UserRepository(api, FakeUserDao())
        repository.refresh()

        api.shouldFail = true
        val result = repository.refresh()

        assertTrue(result.isFailure)
        assertEquals(1, repository.users.first().size)        // still showing the saved user
    }
}
```

Output

```
UserRepositoryTest
  PASSED  refresh failure keeps old data
  PASSED  refresh saves users from the api
2 tests, 0 failed
```

Code	Meaning
= runTest { }	Runs a test that calls suspend functions. (In real kotlinx-coroutines-test, delays are skipped, so tests stay fast)
repository.users.first()	Collects the first value of the Flow: the current list
var shouldFail	Lets each test switch the fake into an "error" mode

3. Where fakes live

Put fakes in **app/src/test/** next to the tests (or in a shared test module in bigger projects). Many teams name them **FakeXxx** or **TestXxx**. Check your project for existing fakes before writing new ones.

4. Mocks with MockK (for reference)

The same idea with a mocking library (needs MockK; shown for reference)

```
val api = mockk<UserApi>()
coEvery { api.fetchUsers() } returns listOf(User(1, "Anjana"))
// ... call the code ...
coVerify(exactly = 1) { api.fetchUsers() }
```

Use mocks when you need to **verify a call happened** ("was the analytics event sent?"). For data sources, fakes are usually clearer.

Practice Exercises

1. Write a **FakeNoteDao** and tests for a NotesRepository: adding a note shows it in the Flow, and deleting removes it.

2. Add a test that refresh() returns failure when the fake API throws, and that the DAO still has the old list.

3. Write a stub **WeatherApi** that always returns 30.0, and test a repository function that maps it to "Hot" (above 28) or "Pleasant".

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. What is a fake?

Q2. Why did we design repositories around interfaces?

Q3. What does runTest do?

Q4. How do you read the current value of a Flow in a test?

Q5. When is a mock better than a fake?

WEEK 14 • DAY 4

Testing ViewModels

Today's goal: Test ViewModel state changes, including coroutines, with a MainDispatcherRule and fake dependencies.

1. The problem: Dispatchers.Main

viewModelScope runs on **Dispatchers.Main**, the Android UI thread. In a local unit test there's no Android, so there's no main thread, and the test crashes. The fix is a small **JUnit rule** that replaces Main with a test dispatcher.

MainDispatcherRule.kt (put it in app/src/test/, write it once per project)

```
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.test.TestDispatcher
import kotlinx.coroutines.test.UnconfinedTestDispatcher
import kotlinx.coroutines.test.resetMain
import kotlinx.coroutines.test.setMain
import org.junit.rules.TestWatcher
import org.junit.runner.Description

class MainDispatcherRule(
    private val testDispatcher: TestDispatcher = UnconfinedTestDispatcher()
) : TestWatcher() {
    override fun starting(description: Description) {
        Dispatchers.setMain(testDispatcher)
    }

    override fun finished(description: Description) {
        Dispatchers.resetMain()
    }
}

fun main() {
    println("MainDispatcherRule compiled")
}
```

Output

```
MainDispatcherRule compiled
```

starting runs before each test and **finished** after each test. Most projects already have this exact class. Look for it before writing your own.

2. Testing a ViewModel

Let's test the TasksViewModel from Week 13. Because it depends on the **ReminderScheduler interface**, we can give it a fake that records what was scheduled:

TasksViewModelTest (uses the MainDispatcherRule above)

```
import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.*
import org.junit.Assert.assertEquals
import org.junit.Assert.assertNull
import org.junit.Assert.assertTrue
import org.junit.Rule
import org.junit.Test

// --- production code (from Week 13) ---
data class Task(val id: Int, val title: String, val isDone: Boolean = false, val remindAtMinutes: Long? = null)
```

```
interface ReminderScheduler {
    fun schedule(task: Task, delayMinutes: Long)
    fun cancel(taskId: Int)
}
data class TasksUiState(val tasks: List<Task> = emptyList(), val reminderFor: Task? = null)

class TasksViewModel(private val scheduler: ReminderScheduler) : ViewModel() {
    private var nextId = 1
    private val _uiState = MutableStateFlow(TasksUiState())
    val uiState: StateFlow<TasksUiState> = _uiState.asStateFlow()

    fun addTask(title: String) {
        if (title.isBlank()) return
        _uiState.update { it.copy(tasks = it.tasks + Task(nextId++, title.trim())) }
    }
    fun toggleDone(task: Task) {
        val updated = task.copy(isDone = !task.isDone)
        if (updated.isDone) scheduler.cancel(task.id)
        _uiState.update { s -> s.copy(tasks = s.tasks.map { if (it.id == task.id) updated else it }) }
    }
    fun setReminder(task: Task, minutes: Long) {
        scheduler.schedule(task, minutes)
        _uiState.update { s -> s.copy(tasks = s.tasks.map { if (it.id == task.id) it.copy(remindAtMinutes = minutes) else it }) }
    }
}

// --- fake ---
class FakeReminderScheduler : ReminderScheduler {
    val scheduled = mutableMapOf<Int, Long>()
    val cancelled = mutableListOf<Int>()
    override fun schedule(task: Task, delayMinutes: Long) { scheduled[task.id] = delayMinutes }
    override fun cancel(taskId: Int) { cancelled.add(taskId) }
}

// --- tests ---
class TasksViewModelTest {
    @get:Rule
    val mainDispatcherRule = MainDispatcherRule()

    private val scheduler = FakeReminderScheduler()
    private val viewModel = TasksViewModel(scheduler)

    @Test
    fun `adding a task shows it in the state`() {
        viewModel.addTask("  Call Amma ")
        assertEquals(listOf("Call Amma"), viewModel.uiState.value.tasks.map { it.title })
    }

    @Test
    fun `blank titles are ignored`() {
        viewModel.addTask("   ")
        assertTrue(viewModel.uiState.value.tasks.isEmpty())
    }

    @Test
    fun `setting a reminder schedules it`() {
        viewModel.addTask("Pay bill")
        val task = viewModel.uiState.value.tasks.first()

        viewModel.setReminder(task, 60)

        assertEquals(60L, scheduler.scheduled[task.id])
        assertEquals(60L, viewModel.uiState.value.tasks.first().remindAtMinutes)
    }

    @Test
    fun `completing a task cancels its reminder`() {
        viewModel.addTask("Pay bill")
        val task = viewModel.uiState.value.tasks.first()

        viewModel.toggleDone(task)

        assertEquals(listOf(task.id), scheduler.cancelled)
        assertTrue(viewModel.uiState.value.tasks.first().isDone)
        assertNull(viewModel.uiState.value.reminderFor)
    }
}
```

Output

```
TasksViewModelTest
  PASSED  adding a task shows it in the state
  PASSED  blank titles are ignored
  PASSED  completing a task cancels its reminder
  PASSED  setting a reminder schedules it
4 tests, 0 failed
```

Code	Meaning
@get:Rule val mainDispatcherRule	Applies the rule to every test in the class
private val viewModel = TasksViewModel(scheduler)	JUnit creates a **new** test class instance (and so a new ViewModel) for every test, so tests don't affect each other
viewModel.uiState.value	Reads the current state directly. Simple and clear for StateFlow
scheduler.scheduled / cancelled	The fake recorded what the ViewModel asked it to do

3. Testing loading and errors

A ViewModel that loads data (uses the MainDispatcherRule above)

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.launch
import org.junit.Assert.assertEquals
import org.junit.Rule
import org.junit.Test

interface QuoteRepository { suspend fun getQuote(): String }

sealed interface QuoteUiState {
    data object Loading : QuoteUiState
    data class Success(val quote: String) : QuoteUiState
    data class Error(val message: String) : QuoteUiState
}

class QuoteViewModel(private val repository: QuoteRepository) : ViewModel() {
    private val _uiState = MutableStateFlow<QuoteUiState>(QuoteUiState.Loading)
    val uiState: StateFlow<QuoteUiState> = _uiState.asStateFlow()

    fun load() {
        viewModelScope.launch {
            _uiState.value = QuoteUiState.Loading
            _uiState.value = try {
                QuoteUiState.Success(repository.getQuote())
            } catch (e: java.io.IOException) {
                QuoteUiState.Error("No internet connection")
            }
        }
    }
}

class FakeQuoteRepository(var fail: Boolean = false) : QuoteRepository {
    override suspend fun getQuote(): String {
        if (fail) throw java.io.IOException()
        return "Small steps every day."
    }
}

class QuoteViewModelTest {
```

```
    @get:Rule
    val mainDispatcherRule = MainDispatcherRule()

    @Test
    fun `initial state is loading`() {
        val viewModel = QuoteViewModel(FakeQuoteRepository())
        assertEquals(QuoteUiState.Loading, viewModel.uiState.value)
    }

    @Test
    fun `load success shows the quote`() {
        val viewModel = QuoteViewModel(FakeQuoteRepository())
        viewModel.load()
        assertEquals(QuoteUiState.Success("Small steps every day."), viewModel.uiState.value)
    }

    @Test
    fun `load failure shows an error`() {
        val viewModel = QuoteViewModel(FakeQuoteRepository(fail = true))
        viewModel.load()
        assertEquals(QuoteUiState.Error("No internet connection"), viewModel.uiState.value)
    }
}
```

Output

```
QuoteViewModelTest
  PASSED  initial state is loading
  PASSED  load failure shows an error
  PASSED  load success shows the quote
3 tests, 0 failed
```

With **UnconfinedTestDispatcher**, the coroutine started by load() runs immediately, so the state is ready on the next line. Data classes and data objects compare by value, so assertEquals works directly on UiState.

Tip: For flows that emit several values over time (for example Loading then Success), many teams use the **Turbine** library: `viewModel.uiState.test { assertEquals(Loading, awaitItem()); assertEquals(Success(...), awaitItem()) }`.

Practice Exercises

1. Write tests for the Week 8 **LoginViewModel**: the error message clears when the email changes, and canSubmit is false for a short password.

2. Write tests for the Week 9 **TimerViewModel**'s setDuration and reset (no waiting needed).

3. Add a test to QuoteViewModelTest: after a failure, calling load() again with the fake fixed shows the quote.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Why does a ViewModel test need MainDispatcherRule?

Q2. What do starting and finished do in the rule?

Q3. How do you read a StateFlow's current value in a test?

Q4. Why do tests not affect each other even though viewModel is a property?

Q5. What does a fake scheduler let you check?

WEEK 14 • DAY 5

UI Tests and Debugging

Today's goal: Write Compose UI tests with semantics and test tags, and debug problems with Logcat, breakpoints and the Layout Inspector.

1. Compose UI tests

UI tests (in **androidTest/**) start your composables on a device or emulator, find elements, interact with them, and check what's on screen. Because Compose screens are stateless (Week 8), you can test them with any UiState.

A Compose UI test (Android only, in app/src/androidTest/...)

```
class CounterScreenTest {

    @get:Rule
    val composeTestRule = createComposeRule()

    @Test
    fun clickingAdd_increasesCount() {
        composeTestRule.setContent {
            MyAppTheme { Counter() }                    // the Week 6 counter
        }

        composeTestRule.onNodeWithText("Count: 0").assertIsDisplayed()
        composeTestRule.onNodeWithText("Add one").performClick()
        composeTestRule.onNodeWithText("Count: 1").assertIsDisplayed()
    }

    @Test
    fun loginButton_disabledUntilFormValid() {
        composeTestRule.setContent {
            LoginScreen(uiState = LoginUiState(email = "anjana@", password = "123"),
                onEmailChange = { }, onPasswordChange = { }, onLoginClick = { })
        }
        composeTestRule.onNodeWithText("Log in").assertIsNotEnabled()
    }
}
```

API	Does
createComposeRule()	Starts an empty activity to host your composable
setContent { }	Shows the composable to test
onNodeWithText("...") / onNodeWithContentDescription("...") / onNodeWithTag("...")	Finds an element
performClick() / performTextInput("...") / performScrollTo()	Interacts with it
assertIsDisplayed() / assertIsEnabled() / assertIsNotEnabled() / assertTextEquals(...)	Checks it

2. Test tags

Giving elements a stable test id

```
@Composable
fun SearchBar(query: String, onQueryChange: (String) -> Unit) {
    OutlinedTextField(
        value = query,
        onValueChange = onQueryChange,
        label = { Text("Search") },
        modifier = Modifier
            .fillMaxWidth()
            .testTag("search_field")          // composeTestRule.onNodeWithTag("search_field")
    )
}
```

Use **testTag** when text isn't a good way to find an element (it changes with language, or many elements have the same text). Good **contentDescriptions** (Week 4) make elements easy to find in tests **and** accessible.

3. Debugging: finding bugs fast

Tool	Use it when	How
Logcat	You want to see what happened, or the app crashed	Filter with **package:mine** or **tag:MyTag**; crashes show **FATAL EXCEPTION**
Breakpoints + Debugger	You need to see values while the code runs	Click the gutter to add a red dot, run with the **Debug** button (bug icon), then step through
Layout Inspector	The UI looks wrong	Tools → Layout Inspector: see the composable tree, sizes and parameters, and recomposition counts
Database Inspector	Data isn't saved as expected	App Inspection → Database Inspector
Network Inspector / logging interceptor	An API call fails	App Inspection → Network Inspector, or OkHttp logs in Logcat

4. Reading a crash (stack trace)

Sketch: A typical crash in Logcat

```
FATAL EXCEPTION: main
Process: com.example.notes, PID: 12345
java.lang.IndexOutOfBoundsException: Index 3 out of bounds for length 3     <- 1. WHAT went wrong
    at java.util.ArrayList.get(ArrayList.java:427)
    at com.example.notes.ui.QuizViewModel.onNextClick(QuizViewModel.kt:42)   <- 2. the first line in YOUR code
    at com.example.notes.ui.QuizScreenKt$QuizScreen$1.invoke(QuizScreen.kt:88)
    ...
```

- **1.** Read the exception type and message: here, a list index that doesn't exist.
- **2.** Find the **first line from your package** (com.example...). Click it: Android Studio opens that exact line (QuizViewModel.kt line 42).
- **3.** Put a breakpoint there, reproduce the problem with the debugger, and inspect the values.
- **4.** After fixing it, write a **test** that would have caught the bug, so it never comes back.

5. Common crashes and their usual cause

Exception	Usual cause
NullPointerException	Using !! on a null value, or a lateinit used too early
IndexOutOfBoundsException	list[i] with an index that doesn't exist
IllegalStateException: ... duplicate key	Two LazyColumn items with the same key
NumberFormatException	toInt() on text that isn't a number (use toIntOrNull())
SecurityException	A missing permission (INTERNET, POST_NOTIFICATIONS...)
NetworkOnMainThreadException	A network call made without a coroutine/suspend
IllegalStateException: Vertically scrollable component was measured with an infinity maximum height	A LazyColumn inside a verticalScroll Column

Practice Exercises

1. Add testTag("add_button") to a button in one of your screens, and write (on paper or in androidTest) a UI test that clicks it.

2. Put a breakpoint in a ViewModel function, run the app in Debug mode, trigger it, and inspect the UiState values. Try Step Over (F8).

3. Cause a crash on purpose (for example "abc".toInt()), find the FATAL EXCEPTION in Logcat, and click the line from your code.

Solutions are in the Answer Key at the end. Try first and look later!

Quick Quiz

Q1. Where do Compose UI tests live?

Q2. How do you find an element without relying on its text?

Q3. In a stack trace, which line should you look at first?

Q4. What does a breakpoint do?

Q5. What should you do after fixing a bug?

WEEK 14 • DAY 6

Weekly Project: Testing the Notes App

Today's goal: Write a real test suite for the Notes app: logic tests, a fake repository and ViewModel tests.

1. What we'll test

Test class	Tests	Kind
NoteLogicTest	preview() and sortNotes()	Pure logic
FakeNotesRepository	(a test double used by the ViewModel tests)	Fake
NotesViewModelTest	add, ignore blank titles, pin sorting, search, delete	ViewModel with coroutines

2. The production code

A slightly simplified version of the Weeks 11 and 12 Notes app, depending only on the **NotesRepository interface**:

Notes.kt (app/src/main/...)

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.launch

data class Note(val id: Int, val title: String, val content: String, val isPinned: Boolean = false)

interface NotesRepository {
    fun observeNotes(): Flow<List<Note>>
    suspend fun save(note: Note)
    suspend fun delete(id: Int)
}

fun Note.preview(maxLength: Int = 40): String =
    if (content.length <= maxLength) content else content.take(maxLength).trimEnd() + "..."

fun sortNotes(notes: List<Note>): List<Note> =
    notes.sortedWith(compareByDescending<Note> { it.isPinned }.thenBy { it.title.lowercase() })

data class NotesUiState(val notes: List<Note> = emptyList(), val query: String = "")

class NotesViewModel(private val repository: NotesRepository) : ViewModel() {
    private val query = MutableStateFlow("")

    val uiState: StateFlow<NotesUiState> = combine(repository.observeNotes(), query) { notes, q ->
        val filtered = if (q.isBlank()) notes else notes.filter { it.title.contains(q, ignoreCase = true) }
        NotesUiState(sortNotes(filtered), q)
    }.stateIn(viewModelScope, SharingStarted.Eagerly, NotesUiState())

    fun onQueryChange(text: String) { query.value = text }

    fun addNote(title: String, content: String) {
        if (title.isBlank()) return
        viewModelScope.launch { repository.save(Note(0, title.trim(), content.trim())) }
    }

    fun togglePin(note: Note) {
        viewModelScope.launch { repository.save(note.copy(isPinned = !note.isPinned)) }
    }

    fun delete(note: Note) {
        viewModelScope.launch { repository.delete(note.id) }
    }
```

```
}

fun main() {
    println(sortNotes(listOf(Note(1, "b", ""), Note(2, "A", "", isPinned = true), Note(3, "a", ""))).map { it.id })
}
```

Output

```
[2, 3, 1]
```

3. Logic tests

NoteLogicTest.kt (app/src/test/...)

```
import org.junit.Assert.assertEquals
import org.junit.Test

class NoteLogicTest {
    @Test
    fun `short content is not shortened`() {
        assertEquals("Milk and eggs", Note(1, "Shopping", "Milk and eggs").preview())
    }

    @Test
    fun `long content is cut with dots`() {
        val note = Note(1, "Plan", "Finish Week 14 testing lessons and write tests for every screen")
        assertEquals("Finish Week 14 testing lessons and write...", note.preview(40))
    }

    @Test
    fun `pinned notes come first, then alphabetical`() {
        val notes = listOf(Note(1, "banana", ""), Note(2, "Zebra", "", isPinned = true), Note(3, "apple", ""))
        assertEquals(listOf(2, 3, 1), sortNotes(notes).map { it.id })
    }
}
```

Output

```
NoteLogicTest
  PASSED  long content is cut with dots
  PASSED  pinned notes come first, then alphabetical
  PASSED  short content is not shortened
3 tests, 0 failed
```

4. The fake repository

FakeNotesRepository.kt (app/src/test/...)

```
import kotlinx.coroutines.flow.*

class FakeNotesRepository : NotesRepository {
    private val notes = MutableStateFlow<List<Note>>(emptyList())
    private var nextId = 1

    override fun observeNotes(): Flow<List<Note>> = notes

    override suspend fun save(note: Note) {
        notes.value = if (note.id == 0) {
            notes.value + note.copy(id = nextId++)
        } else {
            notes.value.map { if (it.id == note.id) note else it }
        }
    }

    override suspend fun delete(id: Int) {
        notes.value = notes.value.filter { it.id != id }
    }
}

fun main() {
    println("FakeNotesRepository compiled")
}
```

Output

```
FakeNotesRepository compiled
```

5. ViewModel tests

NotesViewModelTest.kt (app/src/test/...)

```
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Rule
import org.junit.Test

class NotesViewModelTest {
    @get:Rule
    val mainDispatcherRule = MainDispatcherRule()

    private val repository = FakeNotesRepository()
    private val viewModel = NotesViewModel(repository)

    @Test
    fun `new notes appear in the list`() {
        viewModel.addNote("Shopping", "Milk")
        assertEquals(listOf("Shopping"), viewModel.uiState.value.notes.map { it.title })
    }

    @Test
    fun `blank titles are not saved`() {
        viewModel.addNote("  ", "content")
        assertTrue(viewModel.uiState.value.notes.isEmpty())
    }

    @Test
    fun `pinning moves a note to the top`() {
```

```
        viewModel.addNote("Apple", "")
        viewModel.addNote("Banana", "")
        val banana = viewModel.uiState.value.notes.first { it.title == "Banana" }

        viewModel.togglePin(banana)

        assertEquals("Banana", viewModel.uiState.value.notes.first().title)
    }

    @Test
    fun `search filters by title ignoring case`() {
        viewModel.addNote("Shopping list", "")
        viewModel.addNote("Study plan", "")

        viewModel.onQueryChange("SHOP")

        assertEquals(listOf("Shopping list"), viewModel.uiState.value.notes.map { it.title })
    }

    @Test
    fun `deleting removes the note`() {
        viewModel.addNote("Temporary", "")
        val note = viewModel.uiState.value.notes.first()

        viewModel.delete(note)

        assertTrue(viewModel.uiState.value.notes.isEmpty())
    }
}
```

Output

```
NotesViewModelTest
  PASSED  blank titles are not saved
  PASSED  deleting removes the note
  PASSED  new notes appear in the list
  PASSED  pinning moves a note to the top
  PASSED  search filters by title ignoring case
5 tests, 0 failed
```

The ViewModel uses **SharingStarted.Eagerly** here so the combined state is always up to date in tests without a collector. With WhileSubscribed (the usual choice in the app), tests start a collector first, for example with Turbine or backgroundScope.

6. Running tests on every change: CI

A minimal GitHub Actions workflow: .github/workflows/test.yml (for reference)

```
name: Unit tests
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 17 }
      - run: ./gradlew testDebugUnitTest
```

Your team almost certainly already has CI. When a pull request shows a red cross, open the CI log, find the failing test name, and run that test locally.

7. Project challenges

- Add tests for the Week 11 note editor: opening the editor for an existing note fills in its title and content.
- Write a Room DAO test in androidTest using **Room.inMemoryDatabaseBuilder** (the official way to test DAOs).
- Write a Compose UI test for NotesScreen: with two notes in the UiState, both titles are displayed.
- Find a bug in one of your earlier projects, write a failing test for it first, then fix it (this is test-driven bug fixing).

Week 14 checklist

I can...	Done?
Explain unit, integration and UI tests, and test/ vs androidTest/	[ ]
Write JUnit tests with assertEquals, assertTrue and good names	[ ]
Use Arrange-Act-Assert and test edge cases	[ ]
Write fakes and test repositories with runTest	[ ]
Test ViewModels with MainDispatcherRule	[ ]
Write a Compose UI test and use testTag	[ ]
Use Logcat, breakpoints and the Layout Inspector to debug	[ ]
Read a stack trace and find the line in my code	[ ]
Write the Notes app test suite	[ ]

Quick Quiz

Q1. Why can NotesViewModel be tested without Room?

Q2. What does FakeNotesRepository do when it saves a note with id 0?

Q3. Why use SharingStarted.Eagerly in this ViewModel?

Q4. Which test checks the sorting rules without any ViewModel?

Q5. What does CI do with your tests?

Answer Key

Try every exercise yourself **before** you look here. Your solution may be different and still correct, as long as the output is right!

Day 1: Why Test? Your First Unit Test

Exercise 1

```
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
import org.junit.Test

fun isEven(n: Int) = n % 2 == 0

class IsEvenTest {
    @Test fun `4 is even`() = assertTrue(isEven(4))
    @Test fun `7 is odd`() = assertFalse(isEven(7))
    @Test fun `0 is even`() = assertTrue(isEven(0))
}
```

Output

```
IsEvenTest
  PASSED  0 is even
  PASSED  4 is even
  PASSED  7 is odd
3 tests, 0 failed
```

Exercise 2

```
import org.junit.Assert.assertEquals
import org.junit.Test

fun celsiusToFahrenheit(c: Double) = c * 9 / 5 + 32

class TemperatureTest {
    @Test fun freezing() = assertEquals(32.0, celsiusToFahrenheit(0.0), 0.01)
    @Test fun boiling() = assertEquals(212.0, celsiusToFahrenheit(100.0), 0.01)
    @Test fun body() = assertEquals(98.6, celsiusToFahrenheit(37.0), 0.01)
}
```

Output

```
TemperatureTest
  PASSED  body
  PASSED  boiling
  PASSED  freezing
3 tests, 0 failed
```

Quiz answers

Q1. In app/src/test/.

Q2. test/ runs on the computer (fast); androidTest/ runs on a device or emulator.

Q3. Marks a function as a test for JUnit to run.

Q4. The expected value.

Q5. ./gradlew test

Day 2: Writing Good Tests

Exercise 1

```
import org.junit.Assert.assertFalse
import org.junit.Assert.assertTrue
import org.junit.Test

fun isValidEmail(email: String) =
    email.contains("@") && email.substringAfter("@").contains(".") && !email.contains(" ")

class EmailValidatorTest {
    @Test fun `valid email`() = assertTrue(isValidEmail("anjana@example.com"))
    @Test fun `missing at sign`() = assertFalse(isValidEmail("anjana.example.com"))
    @Test fun `missing dot after at`() = assertFalse(isValidEmail("anjana@example"))
    @Test fun `contains a space`() = assertFalse(isValidEmail("anjana @example.com"))
    @Test fun `empty`() = assertFalse(isValidEmail(""))
}
```

Output

```
EmailValidatorTest
  PASSED  contains a space
  PASSED  empty
  PASSED  missing at sign
  PASSED  missing dot after at
  PASSED  valid email
5 tests, 0 failed
```

Exercise 2

```
import org.junit.Assert.assertEquals
import org.junit.Test

fun calculateTip(amount: Double, tipPercent: Double, roundUp: Boolean): Double {
    var tip = amount * tipPercent / 100
    if (roundUp) tip = kotlin.math.ceil(tip)
    return tip
}

class TipTest {
    @Test fun `15 percent of 1250`() = assertEquals(187.5, calculateTip(1250.0, 15.0, false), 0.001)
    @Test fun `round up`() = assertEquals(188.0, calculateTip(1250.0, 15.0, true), 0.001)
    @Test fun `zero amount`() = assertEquals(0.0, calculateTip(0.0, 20.0, true), 0.001)
}
```

Output

```
TipTest
  PASSED  15 percent of 1250
  PASSED  round up
  PASSED  zero amount
3 tests, 0 failed
```

Exercise 3

```
import org.junit.Assert.assertTrue
import org.junit.Test

fun getGrade(marks: Int): String {
    require(marks in 0..100) { "Marks must be 0-100" }
    return if (marks >= 40) "Pass" else "Fail"
}

class GradeTest {
    @Test fun `marks above 100 throw`() = assertTrue(runCatching { getGrade(150) }.isFailure)
}
```

Output

```
GradeTest
  PASSED  marks above 100 throw
1 tests, 0 failed
```

Quiz answers

Q1. Arrange, Act, Assert.

Q2. When a test fails, the name tells you what behaviour broke.

Q3. Empty input, boundary values (like exactly 8 characters), negative numbers, null.

Q4. They would be slow and give different results depending on the connection.

Q5. Use runCatching and assert isFailure, or assertFailsWith, or @Test(expected = ...).

Day 3: Fakes and Repository Tests

Exercise 1

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Test

data class Note(val id: Int, val title: String)
interface NoteDao { fun observeAll(): Flow<List<Note>>; suspend fun insert(note: Note); suspend fun delete(id: Int) }
class NotesRepository(private val dao: NoteDao) {
    val notes = dao.observeAll()
    suspend fun add(title: String) = dao.insert(Note(0, title))
    suspend fun delete(id: Int) = dao.delete(id)
}
class FakeNoteDao : NoteDao {
    private val table = MutableStateFlow<List<Note>>(emptyList()); private var next = 1
    override fun observeAll(): Flow<List<Note>> = table
    override suspend fun insert(note: Note) { table.value = table.value + note.copy(id = next++) }
    override suspend fun delete(id: Int) { table.value = table.value.filter { it.id != id } }
}

class NotesRepositoryTest {
    @Test fun `adding shows the note`() = runTest {
        val repo = NotesRepository(FakeNoteDao())
        repo.add("Shopping")
        assertEquals(listOf("Shopping"), repo.notes.first().map { it.title })
    }
    @Test fun `deleting removes the note`() = runTest {
        val repo = NotesRepository(FakeNoteDao())
        repo.add("Shopping")
        repo.delete(1)
        assertEquals(0, repo.notes.first().size)
    }
}
```

Output

```
NotesRepositoryTest
  PASSED  adding shows the note
  PASSED  deleting removes the note
2 tests, 0 failed
```

Exercise 2

```
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Assert.assertTrue
import org.junit.Test

interface Api { suspend fun fetch(): List<String> }
class Dao { val table = MutableStateFlow(listOf("old")) }
class Repo(private val api: Api, private val dao: Dao) {
    suspend fun refresh() = runCatching { dao.table.value = api.fetch() }
}

class RefreshFailureTest {
    @Test fun `failure keeps the old list`() = runTest {
        val dao = Dao()
        val failing = object : Api { override suspend fun fetch(): List<String> = throw java.io.IOException() }
        assertTrue(Repo(failing, dao).refresh().isFailure)
        assertEquals(listOf("old"), dao.table.value)
    }
}
```

Output

```
RefreshFailureTest
  PASSED  failure keeps the old list
1 tests, 0 failed
```

Exercise 3

```
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertEquals
import org.junit.Test

interface WeatherApi { suspend fun temperature(): Double }
class WeatherRepository(private val api: WeatherApi) {
    suspend fun feeling(): String = if (api.temperature() > 28) "Hot" else "Pleasant"
}
class StubWeatherApi(private val value: Double) : WeatherApi { override suspend fun temperature() = value }

class WeatherRepositoryTest {
    @Test fun `30 degrees is hot`() = runTest { assertEquals("Hot", WeatherRepository(StubWeatherApi(30.0)).feeling()) }
    @Test fun `24 degrees is pleasant`() = runTest { assertEquals("Pleasant", WeatherRepository(StubWeatherApi(24.0)).feeling()) }
}
```

Output

```
WeatherRepositoryTest
  PASSED  24 degrees is pleasant
  PASSED  30 degrees is hot
2 tests, 0 failed
```

Quiz answers

Q1. A simplified, working implementation of an interface used in tests (for example storing data in a list).

Q2. So tests can pass fakes instead of the real database and API.

Q3. Runs a test that calls suspend functions (and skips delays with the real library).

Q4. flow.first()

Q5. When you need to verify that a specific call happened.

Day 4: Testing ViewModels

Exercise 1

```
import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.*
import org.junit.Assert.assertFalse
import org.junit.Assert.assertNull
import org.junit.Test

data class LoginUiState(val email: String = "", val password: String = "", val errorMessage: String? = null) {
    val canSubmit get() = email.contains("@") && password.length >= 6
}
class LoginViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(LoginUiState())
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()
    fun onEmailChange(e: String) = _uiState.update { it.copy(email = e, errorMessage = null) }
    fun onPasswordChange(p: String) = _uiState.update { it.copy(password = p) }
    fun showError() = _uiState.update { it.copy(errorMessage = "Wrong password") }
}

class LoginViewModelTest {
    private val vm = LoginViewModel()
    @Test fun `typing clears the error`() { vm.showError(); vm.onEmailChange("a@b.com"); assertNull(vm.uiState.value.errorMessage) }
    @Test fun `short password cannot submit`() { vm.onEmailChange("a@b.com"); vm.onPasswordChange("123"); assertFalse(vm.uiState.value.canSubmit) }
}
```

Output

```
LoginViewModelTest
  PASSED  short password cannot submit
  PASSED  typing clears the error
2 tests, 0 failed
```

Exercise 2

```
import androidx.lifecycle.ViewModel
import kotlinx.coroutines.flow.*
import org.junit.Assert.assertEquals
import org.junit.Test

data class TimerUiState(val totalSeconds: Int = 60, val remainingSeconds: Int = 60)
class TimerViewModel : ViewModel() {
    private val _uiState = MutableStateFlow(TimerUiState())
    val uiState: StateFlow<TimerUiState> = _uiState.asStateFlow()
    fun setDuration(s: Int) { _uiState.value = TimerUiState(s, s) }
    fun reset() = _uiState.update { it.copy(remainingSeconds = it.totalSeconds) }
}

class TimerViewModelTest {
    private val vm = TimerViewModel()
    @Test fun `setDuration updates both values`() { vm.setDuration(300); assertEquals(TimerUiState(300, 300), vm.uiState.value) }
    @Test fun `reset restores the total`() { vm.setDuration(30); vm.reset(); assertEquals(30, vm.uiState.value.remainingSeconds) }
}
```

Output

```
TimerViewModelTest
  PASSED  reset restores the total
  PASSED  setDuration updates both values
2 tests, 0 failed
```

Exercise 3

```
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import kotlinx.coroutines.flow.*
import kotlinx.coroutines.launch
import org.junit.Assert.assertEquals
import org.junit.Rule
import org.junit.Test

interface QuoteRepository { suspend fun getQuote(): String }
class FakeQuoteRepository(var fail: Boolean) : QuoteRepository {
    override suspend fun getQuote(): String { if (fail) throw java.io.IOException(); return "Keep going." }
}
class QuoteViewModel(private val repo: QuoteRepository) : ViewModel() {
    val state = MutableStateFlow("")
    fun load() { viewModelScope.launch { state.value = try { repo.getQuote() } catch (e: java.io.IOException) { "Error" } } }
}

class QuoteRetryTest {
    @get:Rule val rule = MainDispatcherRule()
    @Test fun `retry after failure shows the quote`() {
        val repo = FakeQuoteRepository(fail = true)
        val vm = QuoteViewModel(repo)
        vm.load()
        assertEquals("Error", vm.state.value)
        repo.fail = false
        vm.load()
        assertEquals("Keep going.", vm.state.value)
    }
}
```

Output

```
QuoteRetryTest
  PASSED  retry after failure shows the quote
1 tests, 0 failed
```

Quiz answers

Q1. viewModelScope uses Dispatchers.Main, which doesn't exist in local unit tests; the rule replaces it.

Q2. starting sets the test Main dispatcher before each test; finished resets it afterwards.

Q3. viewModel.uiState.value

Q4. JUnit creates a new instance of the test class for each test.

Q5. What the ViewModel asked it to do (what was scheduled or cancelled).

Day 5: UI Tests and Debugging

Quiz answers

Q1. In app/src/androidTest/ (they run on a device or emulator).

Q2. Give it Modifier.testTag("...") and use onNodeWithTag.

Q3. The first line from your own package (your code).

Q4. Pauses the app at that line in Debug mode so you can inspect values and step through code.

Q5. Write a test that would have caught it, so it doesn't come back.

Day 6: Weekly Project: Testing the Notes App

Quiz answers

Q1. It depends on the NotesRepository interface, so a FakeNotesRepository can be used.

Q2. Adds it as a new note with the next id.

Q3. So the combined state is always calculated in tests, even with no collector.

Q4. NoteLogicTest (pinned notes come first, then alphabetical).

Q5. Runs them automatically on every pull request; failures block merging.

> **Next:** When you finish this week, tell your mentor and ask for the **Week 15** material. Great work, Anjana!